

# Εθνικό Μετσόβιο Πολυτεχνείο

*Επιστήμη Δεδομένων και Μηχανική Μάθηση*

## Εξαμηνιαία Εργασία

Διαχείριση Δεδομένων Μεγάλης Κλίμακας

Όνομα / Α.Μ / mail

Νικόλαος Αναστασίου / 03400283 / dsml00283@mail.ntua.gr

Αθήνα, 27 Ιουνίου 2026

# Περιεχόμενα

|                                                  |           |
|--------------------------------------------------|-----------|
| <b>Εισαγωγή</b>                                  | <b>2</b>  |
| <b>Ζητούμενο 1</b>                               | <b>2</b>  |
| <b>Ζητούμενο 2</b>                               | <b>3</b>  |
| Υλοποίηση Query 1 DataFrame, χωρίς UDF . . . . . | 3         |
| Υλοποίηση Query 1 DataFrame, με UDF . . . . .    | 4         |
| Υλοποίηση Query 1, με RDD . . . . .              | 5         |
| Αποτελέσματα Query 1 . . . . .                   | 6         |
| <b>Ζητούμενο 3</b>                               | <b>8</b>  |
| Υλοποίηση Query 2, με DataFrame . . . . .        | 8         |
| Υλοποίηση Query 2, με SQL . . . . .              | 9         |
| Αποτελέσματα Query 2 . . . . .                   | 10        |
| <b>Ζητούμενο 4</b>                               | <b>12</b> |
| Υλοποίηση Query 3, με DataFrame . . . . .        | 12        |
| Υλοποίηση Query 3, με RDD . . . . .              | 13        |
| Αποτελέσματα Query 3 . . . . .                   | 15        |
| <b>Ζητούμενο 5</b>                               | <b>16</b> |
| Υλοποίηση Query 4, με DataFrame . . . . .        | 16        |
| Αποτελέσματα Query 4 . . . . .                   | 18        |
| Αποτελέσματα A (Query 4) . . . . .               | 19        |
| Αποτελέσματα B (Query 4) . . . . .               | 19        |
| Αποτελέσματα A και B Διαγράμματα . . . . .       | 20        |
| <b>Ζητούμενο 6</b>                               | <b>21</b> |
| Στρατηγικές join Query 3 . . . . .               | 21        |
| Στρατηγικές join Query 4 . . . . .               | 22        |

# Εισαγωγή

Στη παρούσα εργασία υλοποιήθηκαν τα Ζητήματα 1 έως 6 σε περιβάλλον Apache Spark και Apache Hadoop, ενώ ο κώδικας για την απάντηση των Queries γράφτηκε σε Python ακολουθώντας τα templates που μας δόθηκαν.

Η υλοποίηση έγινε τόσο σε τοπικό επίπεδο για την ανάπτυξη του κώδικα αλλά και στο απομακρυσμένο cluster που μας δόθηκε πρόσβαση μέσω VPN. Η ανάπτυξη έγινε σύμφωνα με τον οδηγό του [github.com/ikons/bigdata-dsml](https://github.com/ikons/bigdata-dsml).

Αντί να γίνει εγκατάσταση του Spark και του Hadoop στο WSL των Windows έγινε σε native Ubuntu 24.04.4 LTS καθώς αντί για Windows έχω Linux. Η μόνη διαφορά είναι η έκδοσή της Java που αντί για Java OpenJDK 11 που ήταν η προτεινόμενη χρησιμοποιήθηκε η προ εγκατεστημένη δηλαδή η Java OpenJDK 21.

Ολο το υλικό της εργασίας μπορεί να βρεθεί στο Github repo [github.com/nikos230/bigdata-2026](https://github.com/nikos230/bigdata-2026) στο οποίο άρχισε να ανεβαίνει υλικό από την μέρα που μας ανακοινώθηκε στο μάθημα ότι πρέπει να φαίνεται το commit history, μέχρι τότε η δουλειά δεν ανέβαινε εκεί (με σκοπό να ανέβαινε μια φορά στο τέλος).

Σε κάθε Ζητούμενο κάθε υλοποίηση χρονομετρήθηκε. Σε κάθε εκτέλεση ενός script στον server κάποιοι χρόνοι ήταν παρόμοιοι ενώ κάποιες εκτελέσεις είχαν μεγάλες διαφορές π.χ μεγαλύτερες από 10 sec. Το δεύτερο φαινόμενο δεν γνωρίζω γιατί εμφανίστηκε, ενδεχόμενος γιατί θα έτρεχαν πολλές διεργασίες εκείνη την στιγμή στον server ή στο δίκτυο. Σε κάθε περίπτωση κρατήθηκαν οι χρονομετρήσεις από τις πιο πρόσφατες εκτελέσεις των scripts.

## Ζητούμενο 1

Τα δεδομένα που δόθηκαν για την εργασία είναι σε μορφή .csv η οποία αν και είναι εύκολη στη διαχείριση της και μπορεί να διαβαστεί ανοίγοντας το αρχείο με έναν text editor δεν είναι η βέλτιστη για την υποδομή του Spark. Για τον λόγο αυτό τα δεδομένα μετατράπηκαν στην προτεινόμενη μορφή για Spark που είναι η Parquet.

Υπάρχουν πολλές μορφές δεδομένων οι οποίες είναι καταλληλότερες για Spark έναντι του απλού .csv, αρχικά είναι η μορφή .parquet, στην οποία θα μετατραπούν τα δεδομένα της εργασίας, η οποία είναι Columnar δηλαδή αποθηκεύει τα δεδομένα ανά στήλη και όχι ανά γραμμή. Έτσι αμα ένα query ζητάει μόνο 1 στήλη από τις 10 τότε από τον δίσκο διαβάζεται μόνο αυτή η μια στήλη, και με αυτό τον τρόπο πετυχαίνει μεγαλύτερες ταχύτητες read, καλύτερη συμπίεση δεδομένων, υποστηρίζει επίσης αλλαγές στη δομή των δεδομένων και Predicate Pushdown το οποίο παραλήπτει δεδομένα που δεν χρειάζονται κατά την εκτέλεση ενός query αυξάνοντας έτσι ακόμη περισσότερο την ταχύτητα εκτέλεσης.

Ακόμη ένα format κατάλληλο για Spark είναι το ORC (Optimized Row Columnar), το οποίο είναι Columnar σαν το Parquet, και έχει πολλές ομοιότητες. Αναπτύχθηκε για το Apache Hive και λειτουργεί εκεί καλύτερα από ότι στο Spark. Οι διαφορές ανάμεσα στα 2

είναι ότι: Το ORC έχει σαν μικρότερη μονάδα το Row Group ενώ το Parquet έχει ένα Page 8kb, το ORC έχει Column Pruning το οποίο προσπερνάει τα columns τα οποία δεν χρειάζονται για κάποιο query ενώ το parquet έχει column chunks τα οποία είναι ξεχωριστά αποθηκευμένα μέσα στον δίσκο. Επίσης υπάρχουν διαφορές στο φιλτράρισμα των δεδομένων καθώς στο ORC υπάρχουν φίλτρα για min/max, bloom filters και offsets ενώ στο parquet υπάρχουν αντίστοιχα για file, chunk και page.

## Ζητούμενο 2

Στο ζητούμενο 2 θα δημιουργηθούν 3 υλοποιήσεις για το ίδιο Query όπου στην πρώτη θα χρησιμοποιεί μόνο DataFrame χωρίς User Defined Function (UDF), έπειτα θα υλοποιηθεί ξανά με UDF και τέλος θα γίνει μια υλοποίηση με Resilient Distributed Dataset (RDD). Για όλα τα υπο ερωτήματα τα Crime Data συγχωνευτήκαν σε ένα ενιαίο αρχείο .csv και .parquet για την ευκολότερη διαχείριση τους.

### Υλοποίηση Query 1 DataFrame, χωρίς UDF

Ξεκινώντας από το αρχείο DFQ1.py που μας δόθηκε στον οδηγό της εργασίας χρησιμοποιήθηκε σαν template. Φορτώνονται τα δεδομένα με την μέθοδο `spark.read.parquet(args.crimes_path)` αφού πρόκειται για δεδομένα της μορφής spark. Έπειτα γίνεται filter της στήλης "Premis Desc" και παρήχθηκαν οι τιμές ίσες με "STREET". Στη συνέχεια υπολογίζεται ο αριθμός των συνολικών Crimes για τον υπολογισμό του ποσοστού.

Πριν την εκτέλεση του κώδικα που κάνει το φιλτράρισμα για τις ώρες μέσα στην μέρα εκκίνηθηκε ένας `perf_counter` για την χρονομέτρηση των αποτελεσμάτων, έπειτα εκτελέστηκε ο κώδικας όπου για κάποιες τιμές της στήλης "TIME OCC" τις κατηγοριοποιούμε σε Πρωί, Απόγευμα, Βραδυ και Νύχτα με την μέθοδο `.withColumn` δημιουργείται η νέα στήλη, η κατηγοριοποίηση έγινε συμφωνά με τον πίνακα 1

Έπειτα δημιουργείται μια νέα στήλη με όνομα `Day_Part` και κάνουμε `groupBy` της κατηγορίες και για κάθε κατηγορία μετρήθηκε ο αριθμός των Crimes και διαιρέθηκε με τον συνολικό αριθμό για να προκύψει το ποσοστό.

Τέλος εκτυπώνουμε τα αποτελέσματα, σταματάμε τον `perf_counter` και υπολογίζουμε τον χρόνο εκτέλεσης. Ο `perf_counter` πρέπει να σταματήσει μετά το `.show()` καθώς όλα τα προηγούμενα είναι με `lazy loading` και αρα δεν έχουν εκτελεστεί ακόμη. Ο κώδικας παρουσιάζεται στο listing 1

```
1 dataframe = spark.read.parquet(args.crimes_path)
2 street_crimes = dataframe.filter(col("Premis Desc") == "STREET")
3 total_street_crimes = street_crimes.count()
4 start_time = perf_counter()
5 df_no_udf = street_crimes.withColumnn(
6     'Day_Part',
7     when((col("TIME OCC") >= 500) & (col("TIME OCC") <= 1159), "Prwi")
```

```

8      .when((col("TIME_OCC") >= 1200) & (col("TIME_OCC") <= 1659), "Apogeuma")
9      .when((col("TIME_OCC") >= 1700) & (col("TIME_OCC") <= 2059), "Vradi")
10     .otherwise("Nixta")
11 )
12 end_time = perf_counter()
13 result_without_udf = (
14 df_no_udf.groupBy("Day_Part")
15 .count()
16 .withColumnRenamed("count", "Crime_Count")
17 .withColumn("Percentage", round((col("Crime_Count") / total_street_crimes) *
18     100, 2))
19 .orderBy(col("Percentage").desc())
20 )
21 result_without_udf.show()
22 end_time = perf_counter()

```

**Listing 1:** Η υλοποίηση για το DataFrame, χωρίς UDF (Q1\_DF\_without\_UDF.py)

Όλα τα παραπάνω υλοποιήθηκαν στο script Q1\_DF\_without\_UDF.py το οποίο αναπτύχθηκε τοπικά και όταν, πλέον, έτρεχε χωρίς προβλήματα υποβλήθηκε στο απομακρυσμένο server για εκτέλεση μέσω των εντολών που μας δόθηκαν στον οδηγό της εργασίας. Πιο συγκεκριμένα

Το αρχείο ανέβηκε στον server μέσω της εντολής στο listing 2, και εκτελέστηκε με την εντολή στο listing 3.

```

1 hdfs dfs -put -f /home/nikos/bigdata-dsml/code_nikos/Q1_DF_without_UDF.py /user
  /dsml00283/code_nikos/Q1_DF_without_UDF.py

```

**Listing 2:** Μεταφορά script στον server

```

1 spark-submit --conf spark.executor.cores=1 --conf spark.executor.memory=2g --
  conf spark.executor.instances=2 hdfs://hdfs-namenode.default.svc.cluster.
  local:9000/user/$DSML_USER/code_nikos/Q1_DF_without_UDF.py --crimes-path
  hdfs://hdfs-namenode.default.svc.cluster.local:9000/user/$DSML_USER/
  data_parquet/LA_Crime_Data_combined.parquet

```

**Listing 3:** Υποβολή script στο Spark

## Υλοποίηση Query 1 DataFrame, με UDF

Αρχικά φορτώνονται τα δεδομένα και φιλτράρονται οι τιμές της στήλης "Premis Desc" όπως και παραπάνω. Έπειτα ορίζεται η συνάρτηση get\_day\_part η οποία περνάει σαν όρισμα την τιμή TIME\_OCC με την οποία όπως και παραπάνω κατηγοριοποιούμε τις ώρες μέσα στην ημέρα, η συνάρτηση δέχεται σαν όρισμα μια τιμή time\_occ και επιστέφει την κατηγορία στην οποία ανήκει εκείνη η στιγμή μέσα στην μέρα, σύμφωνα με τον πίνακα 1.

Ξεκινάμε τον perf\_counter και έπειτα με την μέθοδο udf() ορίζουμε την συνάρτηση σαν udf, και με την μέθοδο .withColumn() την εφαρμόζουμε στην νέα στήλη "Day\_Part". Έπειτα κάνουμε groupBy και μετράμε τα εγκλήματα ανα κατηγορία και διαιρούμε με τα συνολικά εγκλήματα και τέλος σταματάμε τον perf\_counter αμέσως μετά το .show(). Στην συνέχεια μεταφέρθηκε το script στον server και εκτελέστηκε.

```

1  dataframe = spark.read.parquet(args.crimes_path)
2  street_crimes = dataframe.filter(col("Premis Desc") == "STREET")
3  total_street_crimes = street_crimes.count()
4
5  def get_day_part(time_occ):
6      time_occ = int(time_occ)
7      if 500 <= time_occ <= 1159:
8          return "Prwi"
9      elif 1200 <= time_occ <= 1659:
10         return "Apogeuma"
11     elif 1700 <= time_occ <= 2059:
12         return "Vradi"
13     else:
14         return "Nixta"
15
16  start_time = perf_counter()
17  day_part_udf = udf(get_day_part, StringType())
18  df_with_udf = street_crimes.withColumn("Day_Part", day_part_udf(col("TIME OCC")
19                                     ))
20  result_with_udf = (
21  df_with_udf.groupBy("Day_Part")
22  .count()
23  .withColumnRenamed("count", "Crime_Count")
24  .withColumn("Percentage", round((col("Crime_Count") / total_street_crimes) *
25                                     100, 2))
26  .orderBy(col("Percentage").desc())
27  )
28  result_with_udf.show()
29  end_time = perf_counter()

```

**Listing 4:** Η υλοποίηση για το DataFrame, με UDF (Q1\_DF\_with\_UDF.py)

## Υλοποίηση Query 1, με RDD

Αρχικά φορτώνονται τα δεδομένα και φιλτράρονται οι τιμές της στήλης "Premis Desc" όπως και παραπάνω. Έπειτα ορίζετε η συνάρτηση `get_day_part` η οποία δέχεται σαν όρισμα μια γραμμή απο το DataFrame και απο αυτή την γραμμή φιλτράρετε η στήλη "TIME OCC", στη συνέχεια, όπως και παραπάνω, ανάλογα με την ώρα τη ημέρας γίνεται η κατηγοριοποίηση συμφωνά με τον πίνακα 1. Οι υλοποιήσεις εκτελέστηκαν με 2 executors (1 core, 2GB memory).

Έπειτα ξεκινάμε το `perf_counter` και κάνουμε `map` με την βοήθεια της συνάρτησης `get_day_part`. Στη πράξη η συνάρτηση θα πάει σε κάθε γραμμή του RDD και θα φτιάξει ζεύγη (key, value) που στην περίπτωση μας θα είναι π.χ (Prwi, 1).

Μετά το `map` κάνουμε `reduceByKey(lambda a, b: a + b)` το οποίο κάνει group ολα τα ίδια keys π.χ ολα τα "Prwi" μεταξύ τους, αρα θα έχουμε κάτι σαν (Prwi: 1, 1, 1, , etc,..) και έπειτα το `lambda` πάει σε κάθε group (key, value) και στο τρέχων άθροισμα `a` προσθέτει την επόμενη τιμή `b`. Αρα μετράμε πόσες τιμές έχει το κάθε group.

Στη συνέχεια κάνουμε ακόμη ένα map στα δεδομένα του προηγούμενου αποτελέσματος που είναι πλέον ("Prwi", 10), ("Apogeuma", 12), κτλ,.. (οι τιμές 10 και 12 είναι ενδεικτικές) και για την δεύτερη τιμή που είναι ο αριθμός των εγκλημάτων σε κάθε κατηγορία της μέρας τον διαιρούμε με τον συνολικό αριθμό εγκλημάτων για να υπολογιστεί το ποσοστό και επιπλέον κάνουμε sort με φθίνουσα σειρά, σταματάμε το perf\_counter και εκτυπώνουμε τα αποτελέσματα. Η διαδικασία αυτή μπορεί να γίνει με την μέθοδο lambda x: (x[0], x[1], round((x[1] / total\_street\_crimes) \* 100, 2)), όπου x είναι όλες οι κατηγορίες ("Prwi", "Apogeuma", "Vradi", "Nixta") και x[0] είναι η κάθε κατηγορία απο τις 4 ενώ x[1] είναι η τιμή count. Για την τιμή αυτή την διαιρούμε και κρατάμε 2 δεκαδικά. Τέλος εκτελείτε άλλη μια μέθοδος lambda x: x[2], ascending=False όπου τώρα εκτελείτε ένα sorting με βάση την τιμή count κατά φθίνουσα σειρά.

```
1 dataframe = spark.read.parquet(args.crimes_path)
2 street_crimes = dataframe.filter(col("Premis Desc") == "STREET").rdd
3 total_street_crimes = street_crimes.count()
4
5 def get_day_part(row):
6     time_occ = int(row["TIME OCC"])
7     if 500 <= time_occ <= 1159:
8         return ("Prwi", 1)
9     elif 1200 <= time_occ <= 1659:
10        return ("Apogeuma", 1)
11    elif 1700 <= time_occ <= 2059:
12        return ("Vradi", 1)
13    else:
14        return ("Nixta", 1)
15
16 start_time = perf_counter()
17 counts = street_crimes.map(get_day_part).reduceByKey(lambda a, b: a + b)
18 rdd_results = counts.map(lambda x: (x[0], x[1], round((x[1] /
19     total_street_crimes) * 100, 2))).sortBy(lambda x: x[2], ascending=False).
20     collect()
21 end_time = perf_counter()
```

**Listing 5:** Η υλοποίηση για το RDD, (Q1\_RDD.py)

## Αποτελέσματα Query 1

Παρακάτω παρουσιάζονται τα αποτελέσματα για την κάθε υλοποίηση όπως παρουσιάστηκαν παραπάνω. Αρχικά στον πίνακα 1 παρουσιάζονται οι κατηγορίες ανά ώρα μέρας που επιλέχθηκαν και εφαρμόστηκαν και στις 3 υλοποιήσεις.

Στη συνέχεια, στον πίνακα 2 παρουσιάζεται το αποτέλεσμα του Query 1, το οποίο είναι ίδιο και για τις 3 υλοποιήσεις. Φαίνεται το μεγαλύτερο ποσοστό εγκλημάτων στον δρόμο "STREET" να εμφανίζεται την Νύχτα με ποσοστό 34.08% ενώ το δεύτερο μεγαλύτερο το Βραδύ με ποσοστό 26.92%, δηλαδή με διάφορα 7.16% από το πρώτο. Στη συνέχεια το Απόγευμα παρουσιάζει ποσοστό 21.23% και τέλος το Πρωί εμφανίζει ποσοστό 17.76%.

| Time Start | Time End | Category |
|------------|----------|----------|
| 500        | 1159     | Prwi     |
| 1200       | 1659     | Apogeuma |
| 1700       | 2059     | Vradi    |
| otherwise  | -        | Nixta    |

**Πίνακας 1:** Ταξινόμηση ώρας σε κατηγορίες.

| Day_Part | Crime_Count | Percentage |
|----------|-------------|------------|
| Nixta    | 251094      | 34.08      |
| Vradi    | 198292      | 26.92      |
| Apogeuma | 156432      | 21.23      |
| Prwi     | 130866      | 17.76      |

**Πίνακας 2:** Αποτελέσματα Query 1.

| Υλοποίηση                        | Χρόνος Εκτέλεσης (seconds) |
|----------------------------------|----------------------------|
| DataFrame, without UDF           | 4.5658                     |
| DataFrame, with UDF              | 6.4506                     |
| DataFrame, RDD                   | 11.6403                    |
| 2 executors (1 core, 2GB memory) |                            |

**Πίνακας 3:** Αποτελέσματα Query 1 με DataFrame με UDF, χωρίς UDF και με RDD (parquet).

Τα αποτελέσματα του πίνακα 3 μπορούν να βρεθούν και στον server με τα παρακάτω drive pod names:

- DataFrame, without UDF: q1-df-without-udf-py-0989f19ef08d6384-driver
- DataFrame, with UDF: q1-df-with-udf-py-23820a9ef090c003-driver
- DataFrame, RDD: q1-rdd-py-e863ef9ef093b24a-driver

| Υλοποίηση                        | Χρόνος Εκτέλεσης (seconds) |
|----------------------------------|----------------------------|
| DataFrame, without UDF           | 22.5179                    |
| DataFrame, with UDF              | 10.5150                    |
| DataFrame, RDD                   | 47.8352                    |
| 2 executors (1 core, 2GB memory) |                            |

**Πίνακας 4:** Αποτελέσματα Query 1 με DataFrame με UDF, χωρίς UDF και με RDD (csv).

Τα αποτελέσματα του πίνακα 4 μπορούν να βρεθούν και στον server με τα παρακάτω drive pod names:

- DataFrame, without UDF: q1-df-without-udf-csv-py-d7430d9ef0a40a9a-driver
- DataFrame, with UDF: q1-df-with-udf-csv-py-29ea0c9ef0a75605-driver
- DataFrame, RDD: q1-rdd-csv-py-e153839ef09eb6ad-driver



Απο τα παραπάνω αποτελέσματα, στους πίνακες 3 και 4 γίνονται 2 συγκρίσεις. Αρχικά συγκρίνονται οι χρόνοι εκτέλεσης μεταξύ των διαφορετικών υλοποιήσεων στον πίνακα 3, DataFrame with UDF, DataFrame without UDF και RDD, όπου η υλοποίηση DataFrame, without UDF έχει τον μικρότερο χρόνο εκτέλεσης έπειτα είναι DataFrame, with UDF και τέλος η υλοποίηση με το RDD. Η τελευταία φαίνεται να έχει μεγαλύτερη διαφορά απο τις άλλες 2. Γενικά η υλοποίηση με UDF θα επρεπε και στους 2 πίνακες να έχει τον μεγαλύτερο χρόνο εκτέλεσης καθώς η UDF πρέπει να περασει απο Python σε Scala ή Java για να τρέχει και αυτο προσθετει επιπλεον χρονο στην εκτελεση.

Παρόμοια αποτελέσματα βλέπουμε και τον πίνακα 4 αλλά πλέον η πιο γρήγορη υλοποίηση είναι η DataFrame, with UDF και όχι η DataFrame, without UDF όπως πριν. Επίσης η υλοποίηση RDD έχει πλέον τον μεγαλύτερο χρόνο εκτέλεσης σε σχέση με τις άλλες 2.

Στη συνέχεια παρατηρούμε οτι οι υλοποιήσεις με .csv έχουν κατά πολύ μεγαλύτερο χρόνο εκτέλεσης σε σχέση με τις αντίστοιχες parquet. Η RDD υλοποίηση έχει χρόνο εκτέλεσης ~47 sec με το csv αλλά ~11 sec με την parquet. Αυτό συμβαίνει καθώς το format parquet ευνοεί κατανομημένη εργασία ενώ csv format όχι. Ακόμα μεγαλύτερες διαφορές παρατηρούμε στις υλοποιήσεις DataFrame, without UDF όπου παρουσιάζει χρόνος εκτέλεσης ~22 sec με .csv ενώ μόνο ~4 sec με .parquet, ομοίως για την υλοποίηση DataFrame, with UDF που έχει χρόνο εκτέλεσης ~10 sec με .csv ενώ μόνο ~6 sec με .parquet.

Το RDD έχει τον μεγαλύτερο χρόνο εκτέλεσης και στις 2 υλοποιήσεις γιατί από πίσω τρέχουν συναρτήσεις τις οποίες το Spark δεν μπορεί να κάνει optimize όπως κάνει με τα DataFrames.

## Ζητούμενο 3

Για το ζητούμενο 3 δημιουργήθηκαν 2 υλοποιήσεις, η πρώτη με DataFrame και η δεύτερη με SQL. Και τα 2 queries θα απαντήσουν στο ερώτημα ποιοι είναι οι τρεις μήνες με τα μεγαλύτερα ποσοστά εγκλημάτων κάθε έτους και θα εκτυπωθούν σε αύξουσα σειρά ως προς του μήνες και φθίνουσα ως προς τον αριθμό των εγκλημάτων και την κατάταξη τους μέσα στο κάθε έτος. Επιπλέον οι υλοποιήσεις θα τρέξουν με 2 executors (1 core, 2GB memory).

## Υλοποίηση Query 2, με DataFrame

Για την DataFrame υλοποίηση δημιουργήθηκε ο κώδικας στο listing 6. Αρχικά ξεκάνουμε έναν per\_counter για την χρονομέτρηση και έπειτα προσθέτουμε μια στήλη στο αρχικό DataFrame με όνομα "date" και τύπου timestamp, χρησιμοποιώντας το format του χρόνου όπως φαίνεται παρακάτω. Στη συνέχεια προσθέτουμε μια στήλη year και μια στήλη month χρησιμοποιώντας τις μεθόδους "year()", "month()" και κάνουμε groupBy ανα χρόνο και ανα μήνα. Έπειτα μετρήθηκαν όλα τα εγκλήματα σε κάθε μήνα. Για κάθε χρόνο κάνουμε orderBy() σε φθίνουσα σειρά τον αριθμό εγκλημάτων με την βοήθεια της μεθόδου Window που δημιουργεί ένα "παράθυ-

ρο” για κάθε έτος. Τέλος εφαρμόζουμε ranking για κάθε γραμμή, δηλαδή για κάθε αριθμό εγκλημάτων και κρατάμε μόνο τις γραμμές με rank > 3, και εκτυπώνουμε τα αποτελέσματα και σταματάμε την χρονομέτρηση.

```
1 time_start = perf_counter()
2 df_date = dataframe.withColumn("date", to_timestamp(col("DATE OCC"), "yyyy MMM
   dd hh:mm:ss a" ))
3 df_year_month = df_date.withColumn("year", year("date")).withColumn("month",
   month("date"))
4 month_counts = df_year_month.groupBy("year", "month").agg(count("*").alias("
   crime_total"))
5 win = Window.partitionBy("year").orderBy(col("crime_total").desc())
6 ranked_months = month_counts.withColumn("ranking", rank().over(win))
7 top_three = ranked_months.filter(col("ranking") <= 3)
8 final_result = top_three.select("year", "month", "crime_total", "ranking").
   orderBy(col("year").asc(), col("crime_total").desc(), col("ranking").asc())
9 final_result.show(truncate=False)
10 time_stop = perf_counter()
```

**Listing 6:** Η υλοποίηση για το DataFrame, (Q2\_DF.py)

## Υλοποίηση Query 2, με SQL

Για την SQL υλοποίηση γράφτηκε ο κώδικας στα listing 7, 8, 9, 10. Αρχικά ξεκινάει η χρονομέτρηση και δημιουργείται ένα view του DataFrame καθώς το sql query δε θα μπορεί να δει το ίδιο το DataFrame. Έπειτα γράφεται το SQL Query το οποίο χωρίζεται στα παρακάτω μέρη:

Στο πρώτο μέρος 7 δημιουργούμε 2 στήλες μια year και μια month, με την βοήθεια των μεθόδων year και month από το view crimes. Οι συναρτήσεις year και month παίρνουν η κάθε μια τον χρόνο και τον μήνα.

```
1 dataframe.createOrReplaceTempView("crimes")
2 sql_query = ""
3     WITH ParsedDates AS (
4         SELECT
5             year(to_timestamp(`DATE OCC`, 'yyyy MMM dd hh:mm:ss a')) AS year,
6             month(to_timestamp(`DATE OCC`, 'yyyy MMM dd hh:mm:ss a')) AS month
7         FROM crimes),
```

**Listing 7:** Η υλοποίηση για το SQL, (Q2\_SQL.py) I

Στη συνέχεια, στο listing 8 κάνουμε groupby ανά έτος και ανά μήνα και μετράμε πόσα εγκλήματα έχουν γίνει σε κάθε μήνα και το βάζουμε στην στήλη crime\_total.

```
1 MonthlyCounts AS (
2     SELECT
3         year,
4         month,
5         COUNT(*) AS crime_total
6     FROM ParsedDates
7     GROUP BY year, month),
```

### Listing 8: Η υλοποίηση για το SQL, (Q2\_SQL.py) II

Έπειτα για κάθε γραμμή στη στήλη crime\_total και για κάθε έτος γίνεται ranking ανάλογα με τον αριθμό των εγκλημάτων και αποθηκεύεται στην στήλη ranking. όπως φαίνεται στο listing 9.

```
1 RankedMonths AS (  
2     SELECT  
3         year,  
4         month,  
5         crime_total,  
6         RANK() OVER (PARTITION BY year ORDER BY crime_total DESC) AS ranking  
7     FROM MonthlyCounts),
```

### Listing 9: Η υλοποίηση για το SQL, (Q2\_SQL.py) III

Τέλος κρατάμε τις γραμμές που έχουν rank > 3 και επιστρέφουμε το αποτέλεσμα και σταματάμε την χρονομέτρηση.

```
1     SELECT  
2         year,  
3         month,  
4         crime_total,  
5         ranking  
6     FROM RankedMonths  
7     WHERE ranking <= 3  
8     ORDER BY year ASC, crime_total DESC, ranking ASC  
9     ""  
10 final_result = spark.sql(sql_query)  
11 final_result.show(truncate=False)  
12 time_stop = perf_counter()
```

### Listing 10: Η υλοποίηση για το SQL, (Q2\_SQL.py) IV

## Αποτελέσματα Query 2

| year | month | crime_total | ranking |
|------|-------|-------------|---------|
| 2010 | 1     | 19524       | 1       |
| 2010 | 3     | 18131       | 2       |
| 2010 | 7     | 17857       | 3       |
| 2011 | 1     | 18144       | 1       |
| 2011 | 7     | 19524       | 2       |
| 2011 | 1     | 17284       | 3       |
| 2012 | 1     | 17958       | 1       |
| 2012 | 8     | 17662       | 2       |
| 2012 | 5     | 17504       | 3       |

Πίνακας 5: Αποτελέσματα Query 2

Απο τα αποτελέσματα του πίνακα 5 παρατηρούμε ότι ο μήνας 1, δηλαδή ο Ιανουάριος έχει κάθε χρόνο τα περισσότερα εγκλήματα. Επίσης οι μήνες 7 και 8, δηλαδή οι Ιούλιος και Αύγουστος είναι μέσα στους top 3 μήνες κάθε χρόνο.

| Υλοποίηση | Χρόνος Εκτέλεσης (seconds) |
|-----------|----------------------------|
| DataFrame | 11.8665                    |
| SQL       | 12.3693                    |

4 executors (1 core, 2GB memory)

**Πίνακας 6:** Αποτελέσματα Query 2 με DataFrame και με SQL (parquet).

Οι χρόνοι εκτέλεσης των 2 υλοποιήσεων παρουσιάζονται στον πίνακα 6. Παρατηρήθηκε ότι και για τις 2 υλοποιήσεις ο χρόνος εκτέλεσης ήταν παρόμοιος  $\approx 12$  sec με την SQL να είναι ελάχιστα πιο γρήγορη. Το αποτέλεσμα αυτό είναι αναμενόμενο καθώς και οι 2 υλοποιήσεις, DataFrame και SQL, χρησιμοποιούν το ίδιο φυσικό πλάνο. Για να βρεθεί αυτό, έτρεξα τον κώδικα ξανά στον δικό μου υπολογιστή και προστέθηκε το `final_result.explain(mode="formatted")` το οποίο μας δείχνει το Logical Plan, το Optimized Logical Plan και τέλος το Physical Plan. Στα listings 11 και 12 παρουσιάζονται τα 2 Physical Plans για κάθε υλοποίηση. Παρατηρούμε ότι στο Physical Plan της SQL υλοποίησης έχουμε ένα επιπλέον Projection, αυτή είναι η μόνη διαφορά ανάμεσα στα 2.

```

1 == Physical Plan ==
2 AdaptiveSparkPlan (16)
3 +- Sort (15)
4   +- Exchange (14)
5     +- Filter (13)
6       +- Window (12)
7         +- WindowGroupLimit (11)
8           +- Sort (10)
9             +- Exchange (9)
10              +- WindowGroupLimit (8)
11                +- Sort (7)
12                  +- HashAggregate (6)
13                    +- Exchange (5)
14                      +- HashAggregate (4)
15                        +- Project (3)
16                          +- Project (2)
17                            +- Scan parquet (1)
18 (1) Scan parquet

```

**Listing 11:** Physical Plan για το DataFrame, (Q2\_DF.py)

```

1 == Physical Plan ==
2 AdaptiveSparkPlan (15)
3 +- Sort (14)
4   +- Exchange (13)
5     +- Filter (12)
6       +- Window (11)
7         +- WindowGroupLimit (10)
8           +- Sort (9)
9             +- Exchange (8)

```

```

10         +- WindowGroupLimit (7)
11         +- Sort (6)
12           +- HashAggregate (5)
13             +- Exchange (4)
14               +- HashAggregate (3)
15                 +- Project (2)
16                   +- Scan parquet (1)
17 (1) Scan parquet

```

**Listing 12:** Physical Plan για το SQL, (Q2\_SQL.py)

Τα αποτελέσματα του πίνακα 6 μπορούν να βρεθούν και στον server με τα παρακάτω drive pod names:

- DataFrame: q2-df-py-3cd7e09ef0b71604-driver
- SQL: q2-sql-py-de57479ef0b8ca4a-driver

## Ζητούμενο 4

Για το ζητούμενο 4 υλοποιήθηκε κώδικας όπου υπολογίζει το μέσο ετήσιο κατακεφαλήν εισόδημα για την διετία 2020-2021. Χρησιμοποιήθηκαν τα LA\_income\_2021 και LA\_Census\_Blocks\_2020 datasets. Οι υλοποιήσεις εκτελέστηκαν με 3 executors (1 cores, 2GB memory). Επιπλέον έγινε χρήση της μεθόδου explain έτσι ώστε να βρεθεί ποια στρατηγική ακολούθησε ο catalyst optimizer και στη συνέχεια πραγματοποιήθηκαν πειράματα με διαφορετικές στρατηγικές που μας ζητούνται αργότερα στο Ζητούμενο 6.

## Υλοποίηση Query 3, με DataFrame

Αρχικά υλοποιήθηκε το query με DataFrame όπως παρουσιάζεται στο listing 13. Φορτώνεται το income dataset και γίνονται μερικές αλλαγές όπως αλλαγή της στήλης "Zip Code" σε "ZipCode", έπειτα γίνεται ένα φιλτράρισμα, με τη μέθοδο .filter για κάθε γραμμή της στήλης "Estimated Median Income" καθώς δεν περιέχει πάντα αριθμούς αλλά σε μερικές εγγραφές έχει και λέξεις, αυτά αφαιρούνται. Έπειτα αν υπάρχουν κόμματα, τελείες ή σύμβολα δολαρίου τότε αυτά αντικαθίστανται με το τίποτα δηλαδή με "" έτσι ώστε οι αριθμοί να είναι καθαροί.

Στη συνέχεια δημιουργούνται Schemas για το αρχείο LA\_Census\_Blocks\_2020 καθώς το αρχείο αρχικά ήταν .geojson και όχι .csv άρα η δομή του είναι διαφορετική. Με αυτά είναι εφικτό να διαβαστεί το αρχείο και από αυτό να πάρουμε μόνο τις 2 στήλες που χρειαζόμαστε δηλαδή τις "ZCTA20" και "POP20", αν δε γίνει αυτό τότε κατά την εκτέλεση του script θα πάρουμε το σφάλμα ότι δεν φτάνει η μνήμη ram όταν παρακάτω κάνουμε το join.

Έπειτα εκκινείται ο per\_counter για την χρονομέτρηση και εκτελείται ένα inner join ανάμεσα στα dataframes income και census (το census έχει τα "ZCTA20" και "POP20"). Με το inner κρατάμε τις εγγραφές που έχουν και οι 2 το "Zip Code". Έτσι θα έχουμε ένα συνολικό

μεγάλο dataframe όπου κάθε οικοδομικό τετράγωνο που έχει έναν "Zip Code" θα έχει και μια τιμή εισοδήματος.

Τέλος κάνουμε groupBy για τα "Zip Code" και για κάθε "Zip Code" αθροίζουμε τον πληθυσμό, έπειτα διαιρούμε το συνολικό income κάθε "Zip Code" με τον κάθε πληθυσμό του για να πάρουμε το κατακεφαλήν εισόδημα.

```
1 dataframe_income = spark.read.parquet(args.income_path)
2 dataframe_income = dataframe_income \
3     .withColumnRenamed("Zip Code", "ZipCode") \
4     .filter(F.col("Estimated Median Income").rlike("^[\\$0-9,\\.]+$")) \
5     .withColumn("Income_Num", F.regexp_replace(F.col("Estimated Median Income"),
6     "[\\$,]", "").cast("float")) \
7     .select("ZipCode", "Income_Num")
8 properties_schema = StructType([
9     StructField("ZCTA20", StringType(), True),
10    StructField("POP20", LongType(), True)
11 ])
12 feature_schema = StructType([
13    StructField("properties", properties_schema, True)
14 ])
15 geojson_schema = StructType([
16    StructField("features", ArrayType(feature_schema), True)
17 ])
18 dataframe_blocks = (
19     spark.read
20     .option("multiLine", "true")
21     .schema(geojson_schema)
22     .json(args.census_path)
23 ).selectExpr("explode(features) as features")
24 dataframe_blocks_final = dataframe_blocks.select(
25     F.col("features.properties.ZCTA20").alias("ZipCode"),
26     F.col("features.properties.POP20").alias("Population")
27 ).dropna(subset=["ZipCode", "Population"])
28 time_start = perf_counter()
29 dataframe_joined = dataframe_blocks_final.join(dataframe_income.hint("BROADCAST"), "ZipCode", "inner")
30 dataframe_result = dataframe_joined.groupBy("ZipCode").agg(
31     F.sum("Population").alias("Total_Population"),
32     F.round((F.sum("Income_Num") / F.sum("Population")), 2).alias("Per_Capita_Income")
33 ).orderBy(F.col("Per_Capita_Income").desc())
34 dataframe_result.explain(True)
35 dataframe_result.show(truncate=False)
36 time_stop = perf_counter()
```

**Listing 13:** Η υλοποίηση για το DataFrame, (Q3\_DF.py)

## Υλοποίηση Query 3, με RDD

Για την υλοποίηση με RDD αρχικά φορτώθηκαν τα δεδομένα και έπειτα ορίστηκε η συνάρτηση `income_cleaned` η οποία χρησιμοποιείται για να γίνει map πάνω στα `income` δεδομένα και να καθαριστούν όπως αναφέρθηκε και παραπάνω. Η συνάρτηση επιστρέφει 2 τιμές ε-

να Zip Code και ένα income. Έτσι πλέον τα δεδομένα income είναι έτοιμα. Στη συνέχεια ορίζεται η συνάρτηση `extract_census` με την οποία το census RDD γίνεται map και απο την συνάρτηση επιστρέφονται 2 τιμές, το Zip Code και ο πληθυσμός σε αυτό. Έτσι και το census RDD είναι έτοιμο. Στη συνέχεια, εκκινείται ο `perf_counter` και εκτελούμε την εντολή `.collectAsMap()` στο income RDD για να το φέρουμε πίσω σε έναν υπολογιστή και με το `.broadcast` το περνάμε στη ram του κάθε executor έτσι όταν γίνει στη συνέχεια το `join` δεν θα χρειάζεται να τραβήξει δεδομένα απο πολλούς clusters μαζί. Ορίζεται η συνάρτηση `map_side_join` με την οποία βλέπουμε αν για κάθε γραμμή του census RDD το Zip Code υπάρχει στο income αν υπάρχει τότε η συνάρτηση επιστρέφει (Zip Code, (Population, Income)) όπου το Income αναφέρεται στο κάθε Zip Code ξεχωριστά. Έπειτα κάνουμε `reduceByKey` στο RDD που έχει παραχθεί απο το προηγούμενο map με την lambda έκφραση `lambda a, b: (a[0] + b[0], a[1] + b[1])`. Στην οποία το a είναι το μέχρι τώρα (Population, Income), (για κάθε Zip Code) και το b είναι το επόμενο (Population, Income) και κάθε μέρος του προστίθενται στο συνολικό άθροισμα, δηλαδή το Population με το Population και το Income με το Income. Έτσι τελικά θα έχουμε για κάθε Zip Code τον συνολικό του πληθυσμό και το συνολικό του income. Τέλος ορίζεται η συνάρτηση `calculate_per_capita` με την οποία γίνεται map το RDD που παράχθηκε απο το reduce και για κάθε ξεχωριστό Zip Code θα διαιρέσει το συνολικό του income με τον συνολικό του πληθυσμό για υπολογιστεί το κατακεφαλήν εισόδημα. Τέλος με την μέθοδο `.collect()` εκτελούνται όλα τα παραπάνω.

```

1 income_rdd = spark.read.parquet(args.income_path).rdd
2 census_rdd = (
3     spark.read
4     .option("multiLine", "true")
5     .schema(geojson_schema)
6     .json(args.census_path)
7 ).selectExpr("explode(features) as features").rdd
8 def income_cleaned(row):
9     zip_code = row["Zip Code"]
10    income_str = row["Estimated Median Income"]
11    if not zip_code or not income_str: return None
12    if not re.match(r'^[\$0-9,\.\.]+\$', income_str): return None
13    clean_str = re.sub(r'[\$,]', '', income_str)
14    try:
15        return (str(zip_code), float(clean_str))
16    except ValueError:
17        return None
18 income_rdd_ = income_rdd.map(income_cleaned).filter(lambda x: x is not None)
19 def extract_census(row):
20     try:
21         props = row["features"]["properties"]
22         zip_code = props["ZCTA20"]
23         pop = props["POP20"]
24         if zip_code is not None and pop is not None:
25             return (str(zip_code), int(pop))
26     except (KeyError, TypeError):
27         pass
28     return None

```

```

29 census_rdd = census_rdd.map(extract_census).filter(lambda x: x is not None)
30 time_start = perf_counter()
31 income_dict = income_rdd.collectAsMap()
32 broadcast_income = spark.sparkContext.broadcast(income_dict)
33 def map_side_join(row):
34     zip_code = row[0]
35     pop = row[1]
36     income_map = broadcast_income.value
37     if zip_code in income_map:
38         return (zip_code, (pop, income_map[zip_code]))
39     return None
40 joined_rdd = census_rdd.map(map_side_join).filter(lambda x: x is not None)
41 reduced_rdd = joined_rdd.reduceByKey(lambda a, b: (a[0] + b[0], a[1] + b[1]))
42 def calculate_per_capita(row):
43     zip_code = row[0]
44     total_pop = row[1][0]
45     total_income = row[1][1]
46     per_capita = round(total_income / total_pop, 2) if total_pop > 0 else 0
47     return (zip_code, total_pop, per_capita)
48 final_rdd = reduced_rdd.map(calculate_per_capita)
49 sorted_rdd = final_rdd.sortBy(lambda x: x[2], ascending=False)
50 results = sorted_rdd.collect()
51 time_stop = perf_counter()

```

**Listing 14:** Η υλοποίηση για το RDD, (Q3\_RDD.py)

### Αποτελέσματα Query 3

| ZipCode | Total_Population | Per_Capita_Income |
|---------|------------------|-------------------|
| 93563   | 143              | 57436.36          |
| 91759   | 74               | 28857.41          |
| 93544   | 1126             | 23665.6           |
| 90740   | 11               | 6819.91           |
| 93532   | 2373             | 6298.53           |
| 93553   | 1921             | 5821.73           |

**Πίνακας 7:** Αποτελέσματα Query 3.

Στον πίνακα 7 παρουσιάζονται τα αποτελέσματα του Query 3. Πιο συγκριμένα εμφανίζονται οι Zip Codes μαζί με τον συνολικό πληθυσμό και το κατακεφαλήν εισόδημα, σε φθίνουσα σειρά (για το εισόδημα).

| Υλοποίηση | Χρόνος Εκτέλεσης (seconds) |
|-----------|----------------------------|
| DataFrame | 29.3214                    |
| RDD       | 12.4431                    |

3 executors (1 core, 2GB memory)

**Πίνακας 8:** Αποτελέσματα Query 3 με DataFrame και με RDD (parquet).



Παραπάνω φαίνονται, στον πίνακα 8 οι χρόνοι εκτέλεσης της κάθε υλοποίησης. Η υλοποίηση RDD φαίνεται να είναι κατά  $\approx 11$  sec πιο γρήγορη από την αντίστοιχη DataFrame. Το αποτέλεσμα αυτό είναι αναμενόμενο καθώς το LA\_Census\_Blocks dataset είναι πολύ μεγάλο και έτσι η κατανεμημένη εργασία ευνόησε όλες τις διεργασίες που χρειάστηκαν να γίνουν, όπως το φιλτράρισμα των δεδομένων και ο καθαρισμός τους καθώς και το join.

Τα αποτελέσματα του πίνακα 8 μπορούν να βρεθούν και στον server με τα παρακάτω drive pod names:

- DataFrame: q3-df-py-c3d7599ef0e324c7-driver
- RDD: q3-rdd-py-cd95329ef0db9fbc-driver

## Ζητούμενο 5

Για την υλοποίηση του Query 4 θα βρεθεί ανα αστυνομικό τμήμα ο αριθμός εγκλημάτων που έλαβαν χώρα πλησιέστερα σε αυτό και η μέση απόσταση τους από την τοποθεσίες που σημειώθηκε κάθε έγκλημα. Η υλοποίηση έγινε με DataFrame API και εκτελέστηκε με τα διάφορα Configurations που μας ζητείται από την εκφώνηση. Όσο αφορά τις στρατηγικές που επιλέγει ο optimizer, αναφέρονται αναλυτικά στο Ζήτημα 6.

### Υλοποίηση Query 4, με DataFrame

Για την υλοποίηση του ερωτήματος σε DataFrame γράφτηκε ο κώδικας στο listing 15. Αρχικά φορτώθηκαν τα δεδομένα LA Crime Data και LA Police stations. Έπειτα πρέπει να γίνει φιλτράρισμα και των 2 dataset.

Ξεκινάμε από το LA Crimes, όπου αρχικά κάνουμε select τις στήλες DR\_NO που είναι το ID κάθε εγκλήματος, την LAT και την LON που μας δίνουν σημειοκά την τοποθεσία κάθε μέρος που έγινε κάποιο έγκλημα. Κάνουμε filter τις τιμές των LAT και LON έτσι ώστε να μην έχουν NULL τιμές στα τελικά δεδομένα καθώς και να μην έχουμε τιμές ίσες με μηδέν καθώς αυτό θα σήμαινε ότι οι συντεταγμένες δεν θα ήταν στο LA αλλά κάπου αλλού στη γη και αρα θα ήταν λάθος.

Έπειτα προχωράμε στο dataset με τα LA Police Stations. Σε αυτό θα κάνουμε select μόνο τις στήλες: DIVISION που είναι το όνομα κάθε αστυνομικού τμήματος (οχι το ID όπως πριν) και τις στήλες Y και X που είναι η θέση κάθε τμήματος σημειοκά.

Εκκινούμε τον perf\_counter και κάνουμε το καρτεσιανό γινόμενο ανάμεσα στα 2 DataFrames που έχουμε κάνει filter παραπάνω, δηλαδή αντιστοιχούμε κάθε τμήμα με κάθε έγκλημα και επίσης κάνουμε broadcast το dataframe των stations γιατί είναι πολύ μικρό σε σχέση με το dataframe των crimes. Αυτό συμβαίνει γιατί είναι προτιμότερο να μετακινήσουμε το μικρό

dataframe σε κάθε worker παρά να μετακινούμε και το μεγάλο. Αυτός είναι ο καλύτερος τρόπος να γίνει το συγκεκριμένο join.

Για να βρεθεί η απόσταση κάθε εγκλήματος από κάθε αστυνομικό τμήμα θα χρησιμοποιηθεί η Ευκλείδεια απόσταση (δηλαδή δε θα βρεθεί η απόσταση μέσω του οδικού δικτύου προς κάθε αστυνομικό τμήμα, καθώς αυτό θα ήταν αρκετά πιο περίπλοκο και δεν είναι το αντικείμενο της παρούσας εργασίας) με βάση τα σημεία που υπάρχουν για κάθε έγκλημα και κάθε αστυνομικό τμήμα. Κάθε σημείο είναι ένα ζεύγος  $(\phi, \lambda)$  και στα 2 datasets αυτά είναι γεωγραφικές συντεταγμένες και πρέπει πρώτα να γίνουν προβολικές απο γεωγραφικές γιατί τώρα αναπαριστούν μοίρες πάνω στο ελλειψοειδές και όχι αποστάσεις πάνω στο επίπεδο. Αρα πρέπει να μετασχηματίσουμε τις τα σημεία απο κάθε dataframe σε προβολικά. Για να γίνει αυτό θα μετατρέψουμε τις μοίρες σε rad και έπειτα θα κάνουμε την πράξη  $X = R \cdot \lambda \cdot \cos(\phi_0)$  για κάθε lon, και  $Y = R \cdot \phi$  για κάθε lat όπου R είναι η ακτίνα της γης σε χιλιόμετρα και  $\phi_0$  είναι το μέσο  $\phi$  της περιοχής του LA. Πλέον έχουμε όλα τα σημεία με X, Y αρα μπορούμε να υπολογίσουμε την ευκλείδεια απόσταση  $d = \sqrt{(X_2 - X_1)^2 + (Y_2 - Y_1)^2}$ . Έτσι για κάθε γραμμή θα έχουμε μια νεα στήλη με όνομα "distance" και θα έχει την απόσταση απο κάθε αστυνομικό τμήμα.

Αφού μας ζητείται ο μέσος όρος των εγκλημάτων που έγιναν πιο κοντά σε κάθε τμήμα θα πρέπει να φιλτράρουμε για κάθε τμήμα όλα τα εγκλήματα και να κρατήσουμε μόνο αυτό που έγινε πιο κοντά. Αρα θα ορίζουμε ένα window το οποίο θα πάει σε κάθε DR\_NO και θα κάνει orderBy το distance το οποίο είναι η απόσταση κάθε εγκλήματος απο το εκάστοτε τμήμα. Εφαρμόζουμε το window στο dataframe και κάνουμε rank όλες τις αποστάσεις για καθ τμήμα, κρατάμε μόνο αυτές με rank = 1.

Τέλος εκτυπώνουμε τα αποτελέσματα κάνοντας groupBy ανα αστυνομικό τμήμα και θέτοντας σειρά εμφάνισης φθίνουσα ανα αριθμό περιστατικών και σταματάμε το perf\_counter.

```
1 df_crimes = spark.read.parquet(args.crimes_path)
2 df_stations = spark.read.parquet(args.stations_path)
3 df_crimes_filtered = df_crimes.select(
4     F.col("DR_NO"),
5     F.col("LAT").alias("crime_lat"),
6     F.col("LON").alias("crime_lon")
7 ).filter(
8     (F.col("crime_lat").isNotNull()) & (F.col("crime_lon").isNotNull()) &
9     (F.col("crime_lat") != 0.0) & (F.col("crime_lon") != 0.0)
10 )
11 df_stations_filtered= df_stations.select(
12     F.col("DIVISION").alias("division"),
13     F.col("Y").alias("station_y"),
14     F.col("X").alias("station_x")
15 )
16 time_start = perf_counter()
17 df_all = df_crimes_filtered.crossJoin(F.broadcast(df_stations_filtered))
18 R = 6371000
19 lat1 = F.radians(F.col("crime_lat"))
20 lon1 = F.radians(F.col("crime_lon"))
21 lat2 = F.radians(F.col("station_y"))
```

```

22 lon2 = F.radians(F.col("station_x"))
23 mean_lat = F.radians(F.lit(34.05))
24 crime_x = R * lon1 * F.cos(mean_lat)
25 crime_y = R * lat1
26 station_x = R * lon2 * F.cos(mean_lat)
27 station_y = R * lat2
28 diff_x = station_x - crime_x
29 diff_y = station_y - crime_y
30 dist = F.sqrt(F.pow(diff_x, 2) + F.pow(diff_y, 2))
31 df_dist = df_all.withColumn("distance", dist)
32 window = Window.partitionBy("DR_NO").orderBy("distance")
33 df_nearest = df_dist.withColumn("rank", F.row_number().over(window)).filter(F.
    col("rank") == 1)
34 df_result = df_nearest.groupBy("division").agg(
35     F.round(F.avg("distance"), 3).alias("average_distance"),
36     F.count("DR_NO").alias("#")
37 ).orderBy(F.col("#").desc())
38 df_result.explain(mode="formatted")
39 df_result.show(truncate=False)
40 time_stop = perf_counter()
41 print(f"\nTime elapsed: {time_stop - time_start:.4f} seconds")
42 spark.stop()

```

**Listing 15:** Η υλοποίηση για το DataFrame, (Q4\_DF.py)

## Αποτελέσματα Query 4

Στον πίνακα 9 παρουσιάζονται τα top 10 αποτελέσματα του query 4. Βλέπουμε ότι στο αστυνομικό τμήμα της περιοχής Hollywood εμφανίζονται τα περισσότερα εγκλήματα με αριθμό 224,073 και με μέση απόσταση 2 χιλιόμετρα από το αστυνομικό τμήμα. Έπειτα ο δεύτερος μεγαλύτερος αριθμός εγκλημάτων βρίσκεται στην περιοχή VAN NUYS με αριθμό 208,203 και μέση απόσταση 2.9 χιλιόμετρα. Γενικά παρατηρούμε ότι η μέσες σποστάσεις δεν έχουν μεγάλη διαφορά κυμαίνονται από 900 μέτρα έως 3.8 χιλιόμετρα και επίσης παρατηρείται μεγάλος αριθμός εγκλημάτων σε όλες τις περιοχές.

| Division        | Average Distance | Number of Crimes |
|-----------------|------------------|------------------|
| HOLLYWOOD       | 2074.096         | 224073           |
| VAN NUYS        | 2941.637         | 208203           |
| SOUTHWEST       | 2191.219         | 189162           |
| WILSHIRE        | 2593.032         | 186340           |
| 77TH STREET     | 1716.191         | 170620           |
| NORTH HOLLYWOOD | 2647.367         | 168167           |
| OLYMPIC         | 1729.396         | 162856           |
| PACIFIC         | 3852.065         | 162027           |
| CENTRAL         | 993.285          | 154689           |
| RAMPART         | 1534.281         | 153204           |

**Πίνακας 9:** Αποτελέσματα Query 4.

Τα αποτελέσματα του πίνακα 9 μπορούν να βρεθούν και στον server με το παρακάτω drive pod name:

- DataFrame: q4-df-py-63daff9eeb2931f3-driver

## Αποτελέσματα Α (Query 4)

Αρχικά μας ζητείται να εκτελέσουμε τον κώδικα του Query 4 σε 2 executors με διαφορετικά configurations, τα οποία παρουσιάζονται στον πίνακα 10 μαζί με τα διαφορετικά αποτελέσματα. Ανα ένα configuration διπλασιάζεται ο αριθμός των cores και προσθέτονται 2GB μνήμης RAM.

| Configuration                    | Χρόνος Εκτέλεσης (seconds) | Drive pod Name                   |
|----------------------------------|----------------------------|----------------------------------|
| 2 executors, 1 cores, 2GB memory | 59.85                      | q4-df-py-63daff9eeb2931f3-driver |
| 2 executors, 2 cores, 4GB memory | 41.33                      | q4-df-py-ad8d209eeb263ba8-driver |
| 2 executors, 4 cores, 8GB memory | 20.57                      | q4-df-py-47a30c9eeb300297-driver |

**Πίνακας 10:** Χρόνοι εκτέλεσης για τα διαφορετικά configurations με 2 executors.

Ξεκινώντας από την πρώτη γραμμή του πίνακα 10 έχουμε το configuration με 1 core και 2GB memory, το οποίο έχει χρόνο εκτέλεσης  $\approx 59$  sec, ενώ το αμέσως επόμενο με 2 cores και 4GB memory έχει  $\approx 41$  sec χρόνο εκτέλεσης. Δηλαδή παρόλο που έχει τα διπλάσια resources δεν έχει τον μισό χρόνο. Τέλος, το configuration με τα 4 cores και 8GB memory φαίνεται να είναι το πιο γρήγορο με  $\approx 20$  sec χρόνο εκτέλεσης. Από τα παραπάνω συμπεραίνουμε ότι όσο πιο πολλά cores και GB memory δώσουμε, τόσο καλύτερο χρόνο εκτέλεσης θα έχουμε.

Όλα τα παραπάνω αναφέρονται μόνο για 2 executors. Στη συνέχεια εκτελούνται πειράματα με διαφορετικό αριθμό τόσο απο executors όσο και απο cores και memory. Σε αντίθεση με τα προηγούμενα πειράματα τώρα κάθε configuration θα έχει στο σύνολο 8 cores και 16GB memory αλλά με διαφορετικό τρόπο χωρισμένα.

## Αποτελέσματα Β (Query 4)

Στον πίνακα 11 παρουσιάζονται τα αποτελέσματα για κάθε configuration που μας ζητείται. Στη πρώτη γραμμή έχουμε 2 executors, 4 cores και 8GB memory δηλαδή είναι το ίδιο configuration με την τελευταία γραμμή του πίνακα 10 και βλέπουμε ότι οι χρόνοι εκτέλεσης είναι πολύ κοντά. Έπειτα έχουμε το configuration με 4 executors, 2 cores και 4GB memory δηλαδή αυξάνουμε τον αριθμό των executors αλλά μειώνουμε τον διαθέσιμο αριθμός απο cores και την μνήμη RAM. Το αποτέλεσμα είναι ότι περνουμε τον ίδιο περίπου χρόνο εκτέλεσης με το πρώτο configuration  $\approx 23$  sec. Στο τελευταίο configuration έχουμε 8 executors με μόνο 1 core και 2 GB memory, ο χρόνος εκτέλεσης είναι λίγο μεγαλύτερο απο τα άλλα 2 configurations  $\approx 26$  sec.

Τα παραπάνω αποτελέσματα βλέπουμε ότι ο χρόνος εκτέλεσης ανάμεσα στα διαφορετικά configurations είναι περίπου ο ίδιος, δηλαδή από 22 έως 26 sec. Όμως παρατηρούμε ότι

ο μικρότερος χρόνος βρίσκεται με τους 2 executors και όχι με τους 8. Αυτό συμβαίνει γιατί έχουμε 4 cores και 8GB RAM με τους 2 ενώ το πείραμα με τους 8 έχει λιγότερα resources ανα executor με συνολικά 1 core ανα executor. Αρα οι λιγότεροι αλλά πιο ισχυροί executors είναι καλύτεροι.

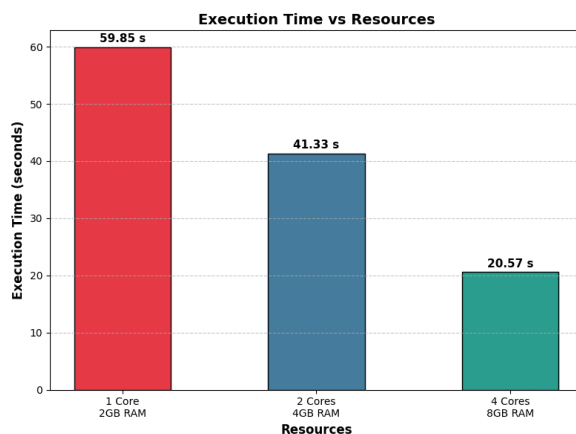
Επίσης μια σημαντική διαφορά όταν έχουμε 2 executors και όταν έχουμε 8 είναι ότι επειδή έχει χρησιμοποιηθεί broadcast join τα δεδομένα του πίνακα stations στέλνονται μόνο 2 φορές όταν έχουμε 2 executors ενώ στέλνονται 8 φορές όταν έχουμε 8 executors, αυτό εξηγεί γιατί ο χρόνος εκτέλεσης είναι μεγαλύτερος με 8 executors έναντι των 2 καθώς τα δεδομένα πρέπει να σταλθούν περισσότερες φορές μέσα από το δίκτυο. Επίσης στον κωδικά γίνεται και ένα groupBy στο τέλος το οποίο εκτελείται τοπικά όταν έχουμε 2 executors ενώ όταν έχουμε 8 θα πρέπει και πάλι να γίνει ανταλλαγή μηνυμάτων μέσω του δικτύου το οποίο προσθέτει overhead.

| Configuration                    | Χρόνος Εκτέλεσης (seconds) | Drive pod Name                   |
|----------------------------------|----------------------------|----------------------------------|
| 2 executors, 4 cores, 8GB memory | 22.03                      | q4-df-py-92bd859eeb331e66-driver |
| 4 executors, 2 cores, 4GB memory | 23.22                      | q4-df-py-5d730d9eeb346822-driver |
| 8 executors, 1 cores, 2GB memory | 26.00                      | q4-df-py-b305039eeb36dd63-driver |

**Πίνακας 11:** Χρόνοι εκτέλεσης για τα διαφορετικά configurations με διαφορετικό αριθμό executors.

## Αποτελέσματα Α και Β Διαγράμματα (Query 4)

Για καλύτερη οπτικοποίηση δημιουργήθηκαν τα παρακάτω 2 σχήματα 1, 2, με τα δεδομένα από τους πίνακες 10, 11.



**Σχήμα 1:** Διάγραμμα χρόνων εκτέλεσης για τα διαφορετικά configurations με 2 executors. **Σχήμα 2:** Διάγραμμα χρόνων εκτέλεσης για διαφορετικό αριθμό executors.

Στο σχήμα 1 παρατηρούμε ότι όσο αυξάνουμε τα resources με τον ίδιο αριθμό executors τόσο γρηγορότερους χρόνους εκτέλεσης πετυχαίνουμε. Ενώ στο σχήμα 2 παρατηρείται ότι οι χρόνοι δεν διαφέρουν πολύ μεταξύ τους αλλά όσο περισσότερους executors προσθέτουμε όσο αυξάνεται ο χρόνος εκτέλεσης όπως αναφέρθηκε και παραπάνω.

## Ζητούμενο 6

Για κάθε Query πρώτα έγινε η υλοποίηση χωρίς την μέθοδο hint έτσι ώστε να δούμε ποιο plan επιλέγει απο μόνος του ο Catalyst Optimizer του Spark το οποίο το βλέπουμε με την μέθοδο explain (βάζοντας σαν mode=formatted). Με την μέθοδο hint, στη συνέχεια, ορίζουμε ρητά με ποια μέθοδο θέλουμε να γίνει το join και ουσιαστικά κάνουμε "overwrite" τον Optimizer και τέλος εκτελούμε τα πειράματα που παρουσιάζονται παρακάτω.

Για κάθε query οι ρυθμίσεις των resources ήταν οι ίδιες προκειμένου να υπάρχει μια ομοιογένεια. Πιο συγκεκριμένα και για το Query 3 και για το Query 4 χρησιμοποιήθηκαν 3 executors με 1 core και 2GB memory. Αυτό το configuration προέρχεται απο το Query 2 όπου είναι και το default που ζητείται στο Ζητούμενο 4. Για το Query 4 μας δίνονται πολλά διαφορετικά configurations στο Ζητούμενο 5 έτσι αποφασίστηκε να χρησιμοποιηθεί το default του Query 4.

### Στρατηγικές join Query 3

Στον πίνακα 12 παρουσιάζονται οι χρόνοι εκτέλεσης του Query 3 με 4 διαφορετικές στρατηγικές join. Το join που εκτελείται στο Query 3 είναι ανάμεσα στα DataFrames: Blocks και Income με κοινό πεδίο το "ZipCode". Επίσης και τα 2 DataFrames είναι μικρά σε μέγεθος, δηλαδή πριν την επιλογή συγκριμένων στηλών τα αρχεία έχουν μέγεθος μερικών kb.

| Στρατηγικές          | Χρόνος Εκτέλεσης (seconds) | Drive pod Name                   |
|----------------------|----------------------------|----------------------------------|
| Broadcast (default)  | 31.74                      | q3-df-py-f923789e445fd227-driver |
| Merge                | 14.02                      | q3-df-py-9a9b8f9e4598778d-driver |
| Shuffle Hash         | 20.80                      | q3-df-py-98a58f9e459a7d8e-driver |
| Shuffle Replicate nl | 26.54                      | q3-df-py-cb9c4b9e459cc00a-driver |

**Πίνακας 12:** Σύγκριση διαφορετικών στρατηγικών join για το Query 3.

Απο τον παραπάνω πίνακα παρατηρούμε οτι το Merge Join έχει το μικρότερο χρόνο εκτέλεσης με διαφορά  $\approx 7$  sec απο τον δεύτερο καλύτερο χρόνο που είναι 20.8 sec του Shuffle Hash. Το Merge Join έχει τον μικρότερο χρόνο γιατί και τα 2 DataFrames είναι πολύ μικρά. Ο τρόπος που λειτουργεί είναι οτι κάθε ενα απο τα DataFrames γίνεται Shuffle και Sort (map) και έτσι κλειδιά με το ίδιο νούμερο να πηγαίνουν στο ίδιο worker και τότε γίνεται το join (reduce), καθώς τα DataFrames είναι πολύ μικρά η διαδικασία του Shuffle είναι πολύ γρήγορη. Αρα αυτή είναι και η πιο καλή μέθοδος για τα συγκριμένα datasets.

Ο δεύτερος μικρότερος χρόνος είναι Shuffle Hash. Το οποίο λειτουργεί με παρόμοιο τρόπο όπως το Merge, δηλαδή αρχικά κάνει Shuffle και στέλνει τα ίδια κλειδιά σε ιδίους workers (map). Στους reducers το μικρότερο απο τα 2 datasets γίνεται hash και περνάει στη RAM, τέλος εκτελείτε το join. Στη περίπτωση μας η δημιουργία του hash table είναι προσθετό overhead καθώς τα datasets είναι πολύ μικρά και αρα μπορούν να γίνουν join σε μικρο χρόνο και χωρίς αυτό, και αρα για αυτό αυτή η μέθοδος είναι η δεύτερη πιο γρήγορη.

Έπειτα ο τρίτος μικρότερος χρόνος είναι του Shuffle Replicate nested loop με διαφορά  $\approx 6$  sec απο το Shuffle Hash. Ο λόγος που είναι αρκετά πιο μεγάλος ο χρόνος εκτέλεσης είναι γιατί στην πράξη εκτελείται ένα καρτεσιανό γινόμενο ανάμεσα στα 2 datasets. Τα δεδομένα αρχικά γίνονται replicate δηλαδή κάθε worker (map) απόκτη τα κομμάτια που χρειάζεται από τα 2 datasets, και αυτό προκαλεί πολύ κίνηση στο δίκτυο αρα αυξάνει το χρόνο εκτέλεσης. Έπειτα στους reducers εκτελείτε το join.

Τέλος ο μεγαλύτερος χρόνος εκτέλεσης βρίσκεται στο Broadcast Join. Η μέθοδος αυτή παίρνει το μικρότερο απο τα 2 datasets και το στέλνει σε κάθε worker, και έπειτα γίνεται το join ανάμεσα στα 2 datasets δηλαδή στο μικρο που είναι ολόκληρο και σε κάθε κομμάτι του μεγάλου dataset. Στη περίπτωση μας και τα 2 datasets είναι πολύ μικρά και αρα αυτή η μέθοδος προσθέτει πάρα πολύ overhead στη διαδικασία του join παρόλο που είναι map-side join.

## Στρατηγικές join Query 4

Στον πίνακα 13 παρουσιάζονται οι χρόνοι εκτέλεσης του Query 4 με 2 διαφορετικές στρατηγικές join. Καθώς απο το ίδιο το Query μας αναγκάζει να χρησιμοποιήσουμε Cross Join, δηλαδή καρτεσιανό γινόμενο, οι μόνες μέθοδοι που μπορούμε να εκτελέσουμε είναι τα BroadcastNestedLoopJoin και το Shuffle Replicate nl.

| Στρατηγικές                       | Χρόνος Εκτέλεσης (seconds) | Drive pod Name                   |
|-----------------------------------|----------------------------|----------------------------------|
| BroadcastNestedLoopJoin (default) | 43.4061                    | q4-df-py-fb0ea79eeef1b737-driver |
| Shuffle Replicate nl              | 45.9918                    | q4-df-py-be7cf59eeef5e74-driver  |

**Πίνακας 13:** Σύγκριση διαφορετικών στρατηγικών join για το Query 4.

Παρατηρούμε ότι οι χρόνοι εκτέλεσης και για τις 2 μεθόδους Join είναι παρόμοιοι. Το BroadcastNestedLoopJoin φαίνεται να είναι 2 sec πιο γρήγορο απο το Shuffle Replicate nl. Αυτό ενδεχόμενος να συμβαίνει γιατί δεν κάνει καθόλου Shuffle απλώς περνάει το μικρότερο απο τα 2 datasets σε όλους τους mappers και ο κάθε mapper διαβάζει το κομμάτι του απο το μεγάλο dataset και έπειτα εκτελείτε το join. Είναι ένα map-side join.

Το Shuffle Replicate nl είναι λίγο πιο αργό καθώς δεν κάνει map-side only join αλλά πρώτα κάνει Shuffle τα δεδομένα δηλαδή στέλνει γραμμές με το ίδιο κλειδί στους ιδίους reducers ενώ στο reducers επίσης στέλνεται και ο μικρότερος απο τους 2 πίνακες, εκεί γίνεται και το join. Αρα αφού υπάρχει και το βήμα του Shuffle (αρα θέλει και reduce) είναι πιο αργό απο την μέθοδο BroadcastNestedLoopJoin.

Απο τις 2 παραπάνω μεθόδους οποία και αν επιλεχθεί το αποτέλεσμα για τα συγκεκριμένα datasets θα είναι το ίδιο, αν έπρεπε να γίνει μια επιλογή αυτή θα ήταν του BroadcastNestedLoopJoin καθώς φαίνεται να έχει ελάχιστα μικρότερο χρόνο εκτέλεσης.